Lec 15 - Mocking

COMPSCI 220 - Perretta

Goals

- Think critically about the difference between unit testing & integration testing
- Understand what "mocking" is and why we need it
- Think critically about the tradeoffs between different ways of applying mocking

Unit vs. Integration Testing

Unit testing:

- Test components in **isolation** (how much isolation?)
  - Dependencies replaced with "mocks" (fake versions)
- Tests are **small** and run **fast**

Integration testing:

- Test interactions between components
  - Real dependency implementation needed
- Relatively fast (depends on dependencies)

Unit Testing: How much isolation?

In groups of 2-3, discuss the following (4 min). Submit to today's activity form.

Should we use a real or fake ("mocked") dependency when testing these things?

- `structureFactory` depends on the `House` class
- A **text editor** reads/writes from/to the **filesystem**
- A web application **checkout page** processes a **credit card payment**
- `Model.addAgent` needs a derived `Agent` class

Some Guidelines

- Is it deterministic?
- Is it expensive?
- Can we inspect the state after an action?
  - e.g., Was an email sent?

A Word of Caution

Too much mocking can make tests **brittle**.

What happens if a dependency changes? (will my test fail?)

How do I know my fake version acts like the real one?

Are my unit & integration tests redundant?

Unit vs. Integration Testing: Where's the line?

Don't get caught up in the details.

Consider the tradeoffs.



"There can be no peace until they renounce their Rabbit God and accept our Duck God."

Scenario (Live Coding)

Board game:

- Players roll dice & move that many spaces
- Something happens depending on square landed on
  - e.g., Earn points, move forward or backward
- Player wins when target score reached